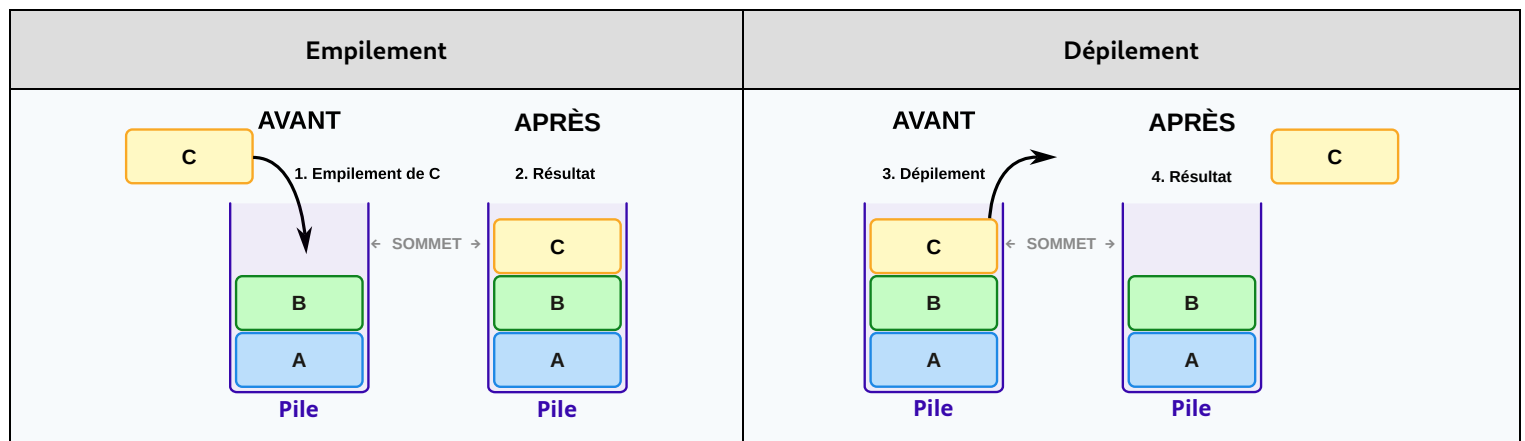


Piles et files

1. Définitions

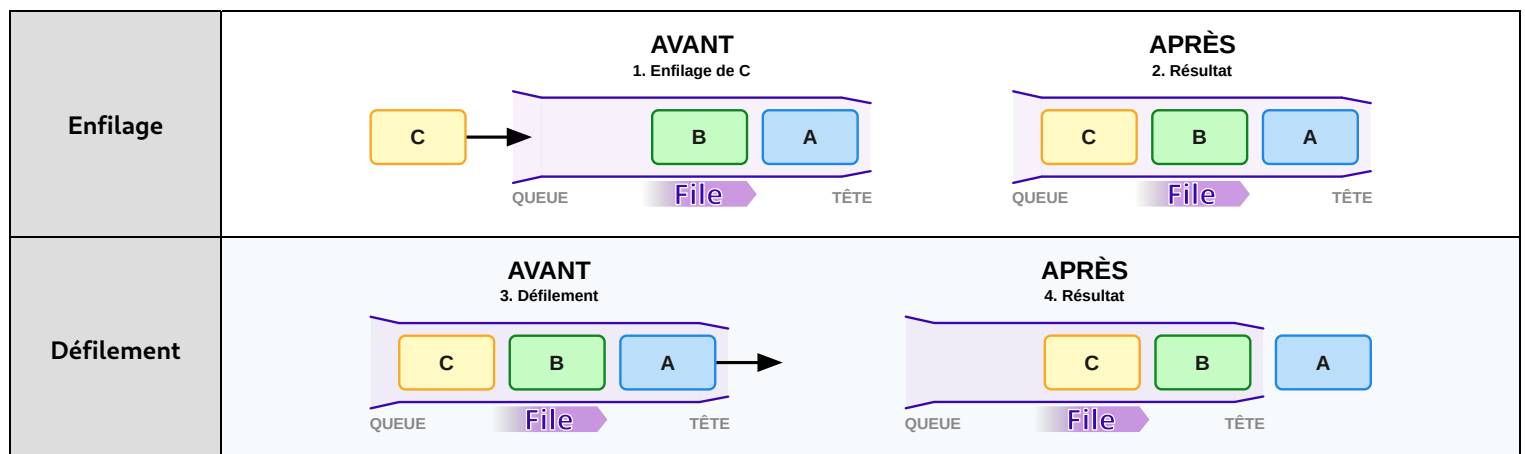
1.1. Pile

Une **pile** (*stack* en anglais) suit le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. On ne peut accéder qu'au *sommet*.



1.2. File

Une **file** (*queue* en anglais) suit le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retiré. On accède à la *tête* (sortie) **et** à la *queue* (entrée).



 **Remarque :** on peut aussi représenter une file avec une progression de la droite vers la gauche ▼

C'est d'ailleurs ce qui se passe logiquement en Python, lorsqu'on utilise le type natif `list` pour simuler (peu efficacement) une file. Voir ci-dessous.

2. Opérations fondamentales

Opération	Pile	File
Ajouter un élément	<code>empiler(x)</code> ou <code>push(x)</code> — au sommet	<code>enfiler(x)</code> ou <code>enqueue(x)</code> — en queue
Retirer un élément	<code>depiler()</code> ou <code>pop()</code> — depuis le sommet	<code>defiler()</code> ou <code>dequeue()</code> — depuis la tête
Consulter <i>sans retirer</i>	<code>lire_sommet()</code> ou <code>peek()</code> — le sommet	<code>lire_tete()</code> ou <code>front()</code> — la tête
Tester la vacuité	<code>est_vide()</code> ou <code>is_empty()</code>	<code>est_vide()</code> ou <code>is_empty()</code>

Toutes ces opérations sont *efficaces* (complexité en $\mathcal{O}(1)$) si la structure est *bien* implémentée.

3. Implémentation Python

3.1. Pile : avec le type list natif de Python

Le type `list` de Python convient à merveille à une utilisation « façon **pile** ». Dans ce contexte, le *sommet* est situé « à *droite* » de la `list` : on *empile* **et** *dépile* donc par la *droite*.

```
pile = []
pile.append(x)    # On empile efficacement (en O(1))
pile.pop()        # On dépile efficacement (en O(1))
pile[-1]          # On consulte le sommet efficacement (en O(1))
```

3.2. File : pourquoi le type list de Python est inadapté

Le type `list` de Python n'est pas optimal pour une utilisation « façon **file** ». Dans ce contexte, la *tête* est situé « à *gauche* » de la `list` : on *enfile* par la *droite*, et on *défile* par la *gauche*.

```
file.append(x)    # On enfile efficacement (en O(1))
file.pop(0)       # On défile de façon sous-optimale (en O(n))
```

Retirer le premier élément d'une `list` force Python à *décaler tous les éléments suivants* d'une position vers la gauche : le coût est proportionnel à la taille n de la structure (donc en $\mathcal{O}(n)$).

3.3. File : deque du module collections

C'est la structure à privilégier dès que l'on simule une file en Python. En effet, une `deque` (*double-ended queue* en anglais) est implémentée comme une liste **doublement chaînée** de blocs : l'ajout et le retrait aux deux extrémités sont garantis en $\mathcal{O}(1)$, sans décalage d'éléments.

```
from collections import deque
file = deque()
file.append(x)    # Enfiler en O(1)
file.popleft()    # Défiler en O(1)
file              # Consulter la tête en O(1)
```

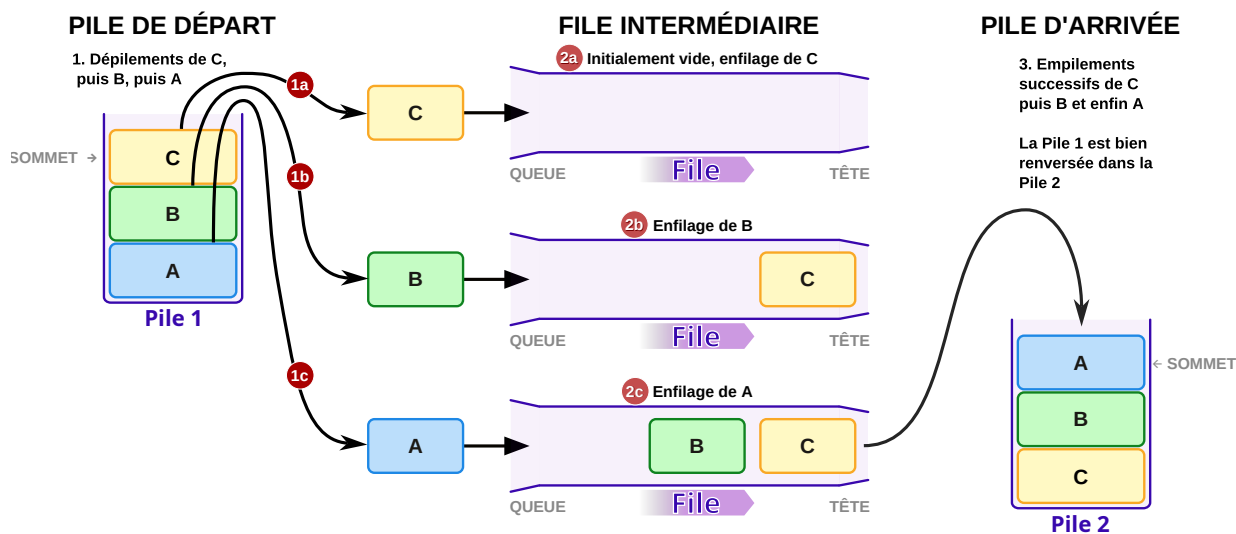
4. Opérations classiques

4.1. Renverser une pile

L'idée : on vide la *pile* dans une *file* (la logique *FIFO* conserve l'ordre d'arrivée) ; puis on ré-empile *depuis* la file — ce qui inverse l'ordre :

```
from collections import deque

def renverser(pile):
    f = deque()
    while pile:
        f.append(pile.pop())      # C sort en premier → tête de file
    while f:
        pile.append(f.popleft()) # C rentre en dernier → fond de pile
```



4.2. Construire une structure avec deux exemplaires de l'autre

Pour créer une file avec deux piles (`p_in` et `p_out`) :

- enfiler de nouveaux éléments se fait en les empilant dans `p_in` ;
- pour défiler, on utilise `p_out`. Si cette pile est vide, on commence par y transférer *tout* `p_in` (ce qui renverse l'ordre des éléments qui y figuraient). Chaque élément n'est transféré qu'*une seule fois* (donc le coût reste en $\mathcal{O}(1)$).

```
class FileAvecDeuxPiles:
    def __init__(self):
        self.p_in = []          # reçoit les nouveaux éléments
        self.p_out = []         # fournit les éléments à défiler

    def enfile(self, x):
        self.p_in.append(x)

    def defile(self):
        if not self.p_out:      # p_out vide : on transfère
            while self.p_in:
                self.p_out.append(self.p_in.pop())
        return self.p_out.pop()
```

Remarque : on peut aussi créer une pile avec deux files ▼

Le principe est symétrique, mais moins élégant. En effet, les action d'empiler ou de dépiler peuvent demander un temps propotionnel à la taille n de la file, dans le pire des cas (on ne peut pas faire mieux avec des files pures).

4.3. Application : vérifier un parenthésage

Une expression est *correctement parenthésée* si chaque parenthèse fermante correspond à la dernière ouvrante non encore fermée. La pile est l'outil naturel pour traiter ce problème : on empile chaque (et on dépile à chaque) ; un dépilement sur pile vide ou une pile non vide en fin de lecture signalent une erreur.

```
def parentheses_ok(expr):
    pile = []
    for c in expr:
        if c == '(':
            pile.append(c)
        elif c == ')':
            if not pile:
                return False          # On a trouvé une ( fermante sans ouvrante correspondante
            pile.pop()
    return len(pile) == 0            # Il reste au moins une ( qui n'a pas été fermée

parentheses_ok("(a + (b * c))")    # Renverra `True`
parentheses_ok("(a + b) * c)")    # Renverra `False` (une parenthèse fermante en trop)
```

5. Récapitulatif

	Pile	File
Principe	LIFO	FIFO
Entrée / sortie	même extrémité	extrémités opposées
Python natif	list	list sous-optimal, contrairement à deque (collections)
Cas d'usage	DFS, récursivité, annulation	BFS, ordonnancement